

Network Intrusion Detection System Using Machine Learning on CIC-IDS2017 Dataset

Team Leader: Nguyen Hoang An

Members: Vu Ngoc Hai Dang

Le Trung Kien

Le Trung Hieu

Do Anh Thu

Advisor: *Mr. Luong Trung Kien*

Abstract: Network intrusion detection is a critical component of modern cybersecurity infrastructure. As network attacks become increasingly sophisticated and voluminous, traditional signature-based detection systems struggle to keep pace. This project presents a machine-learning-based Network Intrusion Detection System (NIDS) built on the CICIDS2017 benchmark dataset, which comprises approximately 2.83 million network flow records covering both benign traffic and 14 distinct attack types. A comprehensive data preparation pipeline was developed, including duplicate removal, infinite-value handling, label standardisation (grouping 15 raw labels into 8 categories), and data-driven feature selection using both statistical tests (Kruskal–Wallis) and model-based importance (Random Forest). XGBoost achieved the highest F1-macro of 0.8585. The final model was deployed as an interactive Streamlit dashboard for real-time classification. Overall, the project demonstrates that a well-engineered data pipeline combined with ensemble tree-based classifiers can achieve robust multi-class intrusion detection performance.

Keywords: Network Intrusion Detection; NIDS; CICIDS2017; Machine Learning; XGBoost; Feature Selection; Cybersecurity

1 Introduction

The rapid proliferation of devices connected to the Internet and the exponential growth of network traffic have made cybersecurity one of the most pressing challenges of the digital era. According to recent industry reports, the frequency and sophistication of cyberattacks continue to escalate, with network intrusions causing billions of dollars in annual losses [1]. As a result, Network Intrusion Detection Systems (NIDS) have become an indispensable layer of defense in organizational security architectures.

Traditional NIDS rely on signature-based methods, where pre-defined patterns of known attacks are matched against incoming traffic. While effective for detecting well-known threats, these approaches inherently fail to identify novel or zero-day attacks, as they depend on a continuously updated signature database [2]. This limitation has motivated the research community to explore machine learning (ML) approaches, which can learn complex traffic patterns directly from data and generalise to previously unseen attack variants [3].

The CICIDS2017 dataset, published by the Canadian Institute for Cybersecurity, has emerged as one of the most widely used benchmark datasets for evaluating ML-based NIDS. It contains approximately 2.83 million labelled network flow records, generated in a controlled environment that simulates realistic modern-day network traffic and a diverse set of attack scenarios including Denial-of-Service (DoS/DDoS), port scanning, brute force attacks, web-based exploits, botnet activity, and the Heartbleed vulnerability [4]. Each flow is characterised by 78 network-level features extracted using the CICFlowMeter tool, covering metrics such as packet lengths, inter-arrival times, flow durations, and TCP flag counts.

This project tackles the multi-class network intrusion detection problem on the CICIDS2017 dataset: given a set of network flow features as **input**, the system must produce a classification label (**output**) indicating whether the traffic is benign or belongs to one of seven attack categories. The specific objectives are:

- (i) Design and implement a data preprocessing and feature engineering pipeline that handles the dataset's inherent quality issues (missing values, infinite values, duplicate records, and redundant features).
- (ii) Conduct exploratory data analysis (EDA) to understand the dataset's statistical properties, class distribution, and inter-feature relationships.
- (iii) Train, tune, and compare multiple ML classifiers (Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and Extra Trees) with appropriate handling of class imbalance.
- (iv) Deploy the best-performing model as an interactive web-based dashboard for demonstration purposes.

The motivation for choosing this topic is threefold. First, network security directly impacts every organization and individual connected to the internet, making it a topic of significant real-world importance. Second, the CICIDS2017 dataset provides a realistic and well-labeled benchmark that enables reproducible research [4]. Third, the multi-class nature of the problem (rather than simple binary anomaly detection) introduces challenging class-imbalance issues that require careful data engineering and modelling strategies, offering a rich learning opportunity for applied machine learning.

2 Related Works

This section reviews recent published works that employ machine learning techniques for network intrusion detection, with a particular focus on studies that utilize the CICIDS2017 dataset or similar flow-based datasets.

Interpretable Machine Learning Framework for Anomaly Detection in Intrusion Detection Systems using XAI Techniques

Building a lightweight framework using Decision Tree (DT) and Random Forest (RF) models integrated with Explainable AI (XAI) techniques, specifically SHAP and LIME in order to enhance transparency in cybersecurity. These tools are used to provide global feature importance and local decision reasoning, helping security analysts understand why specific network traffic is flagged as an anomaly. For the dataset and data preprocessing, the authors utilized the NSL-KDD dataset, an improved version of KDD'99 that addresses redundancy and class imbalance. Preprocessing included one-hot encoding for categorical attributes, min-max normalization for numeric features, and data cleaning to remove duplicate records. Regarding the results, the Random Forest model achieved the highest performance with an accuracy of approximately 98.1%, a precision of 96.4%, and an AUC-ROC of 0.99. The Decision Tree model also performed competitively at 96.2% accuracy [5]. The XAI analysis identified key features such as service type, source/destination counts (srv_count), and time-to-live (sttl) as major influencers in detection decisions. However, the paper identifies several limitations, primarily involving ethical and security trade-offs. There is a risk that XAI could lead to privacy leaks of sensitive user metadata or be abused by adversaries who might use the provided explanations to sidestep detection mechanisms. Additionally, the study notes that models may still be susceptible to bias if training data overrepresents specific network behaviors.

A Learning Approach for The Identification of Network Intrusions Based on Ensemble XGBoost Classifier

The paper proposes a robust framework to enhance the detection of cyber threats in network traffic,. The main approach of this study is centered on the XGBoost ensemble learner, chosen for its speed and accuracy in parallel tree boosting,. To optimize the model, the authors employed an ensemble feature selection method using XGBoost's feature importance scores to reduce the original 78 attributes down to 34 relevant features, thereby decreasing model complexity and training time,. Furthermore, the One Versus Rest (OVR) technique was used to convert the multi-class dataset into a binary classification problem to distinguish between benign and malicious activities. For the dataset and data preprocessing, the researchers utilized the CICIDS2017 dataset, which is favored for containing modern, representative attack types such as DoS, Botnet, and DDoS,. The preprocessing phase included Exploratory Data Analysis

(EDA) to understand the data properties, followed by feature encoding to convert categorical variables into numerical formats,. Additionally, min-max scaling was applied to normalize the features into a interval, ensuring the learning classifier could process the data effectively. The results obtained from the experimentation across eight different data captures were highly successful, achieving an average accuracy of 98%,. Other performance metrics were equally strong, with precision, recall, and F1-score all reaching 0.98, and an AUC-ROC score of 0.99 [6]. Despite these high performance figures, some limitations can be noted. While the study emphasizes building an "interpretable model" through feature importance, it does not incorporate advanced Explainable AI (XAI) tools like SHAP or LIME for detailed local decision reasoning. Additionally, the use of the OVR technique simplifies the detection into a binary "benign vs. malicious" classification, which might limit the system's ability to provide granular identification of specific attack categories in a single-model output.

Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability

The core idea is to combine multiple Machine Learning (ML) models—such as XGBoost, CatBoost, Random Forest, and Multilayer Perceptron (MLP)—with Explainable AI (XAI) methods, including SHAP, LIME, and ELI5, to generate both global and local interpretations of detection outcomes. In terms of data handling, the research employs the UNSW-NB15 dataset, which includes more than 2.5 million instances across nine attack categories. The preprocessing pipeline is thorough, consisting of duplicate removal, feature selection using Recursive Feature Elimination (RFE), normalization through Min-Max scaling, and class balancing with SMOTE. The experimental findings show that XGBoost and CatBoost deliver the best performance, each reaching an accuracy of 87% while maintaining a low false positive rate of 0.07. Importantly, the XAI techniques highlight Source-to-Destination Time-to-Live (sttl) and Destination Service Count (ct_srv_dst) as the most influential features in identifying malicious traffic. Despite these promising results, the study acknowledges several limitations, including increased computational overhead from XAI integration, dependence on a single dataset (UNSW-NB15), and the possibility of false negatives, which may result in undetected advanced attacks [7].

RADAR: A TTP-based Extensible, Explainable, and Effective System for Network Traffic Analysis and Malware Detection

RADAR's core idea is to leverage the MITRE ATT&CK framework to identify attacker behaviors (TTPs) directly from network traffic. It adopts a modular "divide-and-conquer" design, where each TTP is analyzed using a separate Decision Tree classifier [8]. This model choice enhances explainability, as analysts can trace the exact logical rules and network features behind each prediction. The system then applies aggregation strategies (P1, P2, or P3) to de-

termine whether an overall sample should be classified as malicious. For evaluation, RADAR is tested on a large-scale real-world dataset containing more than 2.28 million samples and around 85 million network flows, collected from sources such as Ember, MalRec, MalShare, and VirusShare. During preprocessing, raw PCAP data is transformed into NetFlow records using YAF (Yet Another Flowmeter), ensuring a lightweight and payload-independent pipeline. These flows are stored in a Neo4j database, where feature-based and heuristic rules are applied to identify specific TTPs. The results show that RADAR achieves strong performance, with an AUC of 0.868, comparable to modern black-box models. It also maintains a false positive rate below 0.2 while delivering high detection accuracy, clearly outperforming traditional rule-based systems. However, the study notes several limitations. Some TTPs—such as the use of uncommon ports—are common in both benign and malicious traffic, meaning TTP detection alone is not sufficient without machine learning support. Additionally, variations in TTP distributions across datasets require environment-specific tuning. Lastly, although the framework is extensible, it currently supports only a limited set of TTPs tailored to NetFlow-based analysis.

In summary, while existing studies demonstrate strong classification performance on the CIDS2017 dataset, several common limitations persist: many works do not adequately address the severe class imbalance present in the dataset, some apply feature scaling or resampling before the train-test split (introducing data leakage), and few provide a deployable end-to-end system. This project addresses these gaps by implementing a carefully ordered preprocessing pipeline that avoids data leakage, using robust statistical methods for feature selection, and deploying the final model through an interactive Streamlit dashboard.

3 Data Preparation

3.1 Dataset Overview

The CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) dataset is one of the most comprehensive and widely-cited benchmark datasets for network intrusion detection research. It was generated by the Canadian Institute for Cybersecurity at the University of New Brunswick and is publicly available on Kaggle and the CIC website. The dataset captures five days of network activity (Monday to Friday), where Monday contains only benign traffics and the subsequent days include various simulated attacks injected into realistic background traffic.

The raw data is distributed across **8 CSV files**, one per day and attack scenario. When concatenated, the combined dataset comprises **2,830,743 rows** and **79 columns** (78 features + 1 label column), totalling approximately 1.7 GB in memory.

3.2 Data Cleaning Pipeline

A systematic preprocessing pipeline was designed and implemented to address several data quality issues inherent in the raw dataset. The pipeline is executed sequentially to ensure each step builds on a clean foundation.

3.2.1 Column Name Standardisation

Leading and trailing whitespace characters were stripped from all column names using `str.strip()`. This step is necessary because the dataset introduces inconsistent spacing in column headers, which can lead to silent errors when accessing columns programmatically.

3.2.2 Duplicate Removal

The dataset was found to contain **308,381 duplicate rows** (10.89% of total records). These duplicates arise because identical network flows can appear across overlapping capture windows. All duplicate rows were removed while retaining the first occurrence, reducing the dataset from 2,830,743 to **2,520,762 rows**.

3.2.3 Identical Column Detection and Removal

A pairwise column comparison was performed and revealed **12 redundant columns** that were exact copies of other columns. Table 1 summarises the identified relationships:

Table 1: Identical columns detected in the CICIDS2017 dataset.

Retained Column	Identical (Dropped) Column(s)
Total Fwd Packets	Subflow Fwd Packets
Total Backward Packets	Subflow Bwd Packets
Fwd PSH Flags	SYN Flag Count
Bwd PSH Flags	Bwd URG Flags, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, Bwd Avg Bulk Rate
Fwd URG Flags	CWE Flag Count
Fwd Header Length	Fwd Header Length.1

After removing these 12 redundant columns, the dataset was reduced from 79 to **66 columns**.

3.2.4 Infinite and Missing Value Handling

Two numeric columns (Flow Bytes/s and Flow Packets/s) contained **infinite values** caused by division-by-zero when the flow duration is zero. **2,775 missing values** infinite values were first replaced with NaN. The column Flow Bytes/s also contained **1,211 missing values** (0.05% of total records). Given the negligible proportion, all rows containing NaN values were dropped, which is a conservative but justifiable strategy for a dataset of this size.

3.3 Label Standardisation and Grouping

The raw Label column contained 15 distinct class labels. To reduce label noise and improve model learnability, semantically related attack types were grouped into broader categories. The label column was also renamed to Attack Type for clarity. Table 2 presents the mapping:

Table 2: Attack type grouping applied to the CICIDS2017 labels.

Group	Original Label(s)	Count
BENIGN	BENIGN	2,095,057
DoS	DoS Hulk, DoS GoldenEye, DoS slowloris, DoS Slowhttptest	193,745
DDoS	DDoS	128,014
PortScan	PortScan	90,694
Brute Force	FTP-Patator, SSH-Patator	9150
Web Attack	Brute Force, XSS, SQL Injection	2,143
Bot	Bot	1,948
Heartbleed	Heartbleed	11
<i>Removed</i>	Infiltration	36

The *Infiltration* class (only 36 samples) was removed because it is too rare to support meaningful learning or evaluation. After label grouping and filtering, the final dataset for analysis contains **8 classes** (including BENIGN).

3.4 Data-Driven Feature Selection

Beyond removing identical columns during cleaning, a data-driven feature selection process was conducted to further reduce dimensionality and remove uninformative features.

3.4.1 Correlation Analysis

A Pearson correlation matrix was computed across all remaining numeric features. Feature pairs with extremely high correlation ($|r| \geq 0.95$) were identified, and one feature from each pair was removed to mitigate multicollinearity. The following **5 features** were dropped based on this criterion:

- Total Backward Packets (redundant with forward packet counts)
- Total Length of Bwd Packets (highly correlated with other length metrics)
- Subflow Bwd Bytes (identical information to backward byte counts)
- Avg Fwd Segment Size (equivalent to Fwd Packet Length Mean)
- Avg Bwd Segment Size (equivalent to Bwd Packet Length Mean)

Figure ?? displays the Pearson correlation matrix for the numeric features, capturing pairs with extremely high correlation.



3.4.2 Statistical and Model-Based Feature Importance

Prior to feature ranking, Levene’s test was applied to assess variance homogeneity across attack-type groups for each feature. Results confirmed that nearly all features exhibit significantly unequal variances ($p \leq 0.05$), validating the use of non-parametric methods rather than classical ANOVA.

Two complementary approaches were then used to evaluate feature importance:

1. **Kruskal–Wallis H-test** (non-parametric): Since the data does not follow a Gaussian distribution and contains many outliers, the non-parametric Kruskal–Wallis test was chosen over ANOVA. This test evaluates whether the distributions of a feature differ significantly across the attack-type classes. Features with very low H-statistics indicate poor discriminative power.
2. **Random Forest feature importance**: A Random Forest classifier (100 trees, max depth 20) was trained and its Gini-based feature importances were extracted. This model-based approach captures non-linear relationships that statistical tests may miss. The Random Forest achieved a cross-validation score of approximately 0.99, confirming its reliability for importance ranking.

By combining results from both methods, **8 features** with consistently low importance scores were removed:

- ECE Flag Count, RST Flag Count, Fwd URG Flags, Idle Std,
- Fwd PSH Flags, Active Std, Down/Up Ratio, URG Flag Count

Figure 1 shows the Kruskal-Wallis H-statistic for each feature.

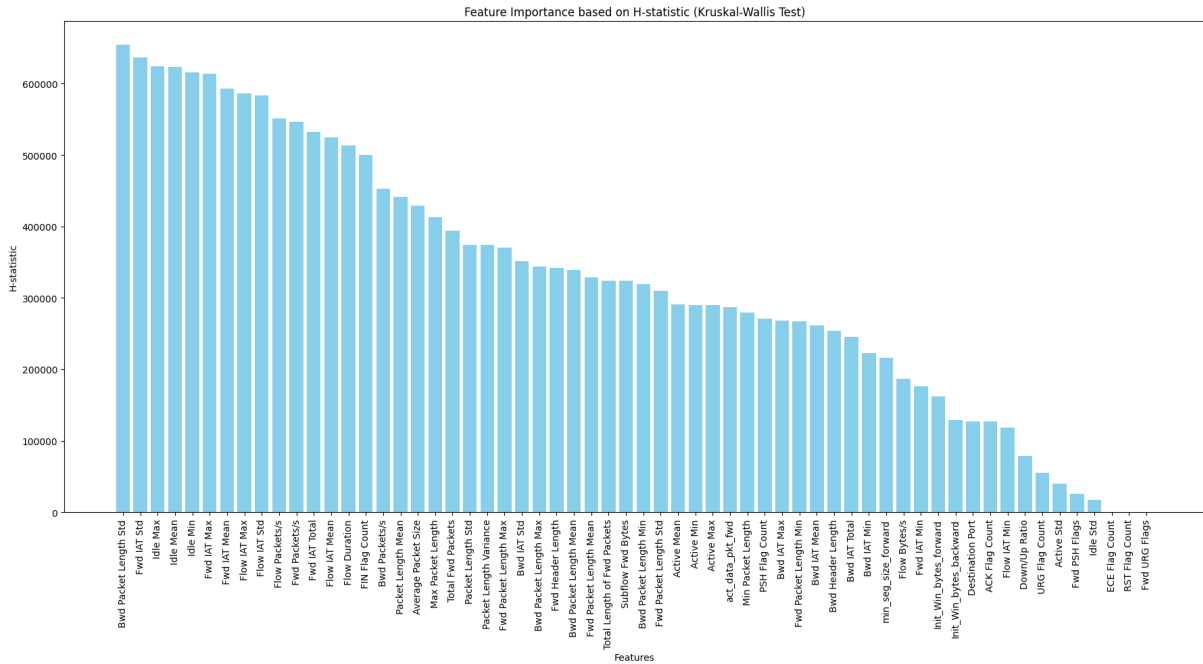


Figure 1: Kruskal-Wallis H-statistic bar chart

Figure 2 ranks the features based on the Gini importances extracted from the Random Forest model.

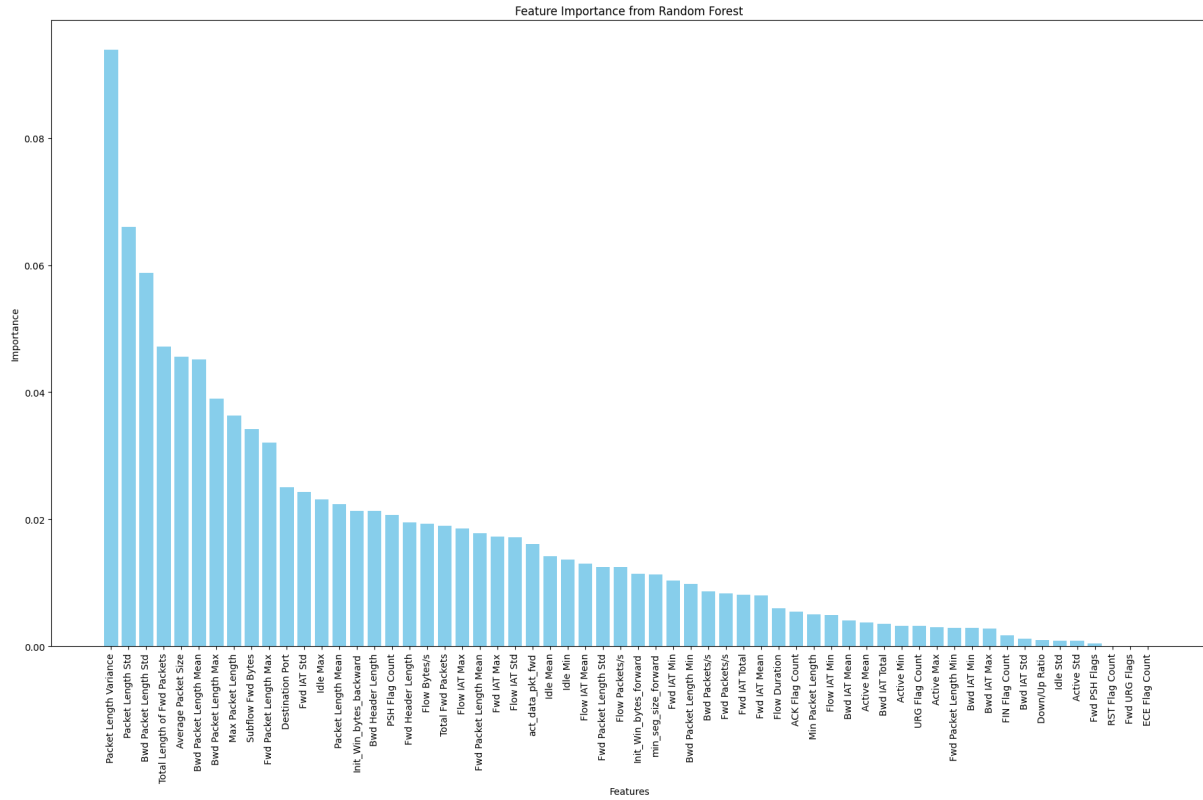


Figure 2: Random Forest feature importance bar chart

3.5 Final Dataset Summary

After the complete preprocessing and feature engineering pipeline, the final cleaned dataset used for model training has the following characteristics:

- **Rows:** $\approx 2,520,000$ (after duplicate/NaN/class removal)
- **Columns:** 54 features + 1 target (Attack Type)
- **Classes:** 7 attack types + BENIGN (8 total; Infiltration and Heartbleed removed due to extreme rarity)
- **Class imbalance:** BENIGN traffic dominates at $\approx 83.1\%$, followed by DoS ($\approx 7.8\%$) and DDoS ($\approx 5.1\%$)

The complete data pipeline is implemented in `notebooks/01_data_pipeline.ipynb`, which orchestrates the modular source code in `src/` and outputs the final `cicids2017_cleaned.csv`.

An important methodological note: techniques such as SMOTE (for resampling) and RobustScaler (for normalisation) were *not* applied at this stage. Applying these techniques to the entire dataset before the train-test split would violate the principle of keeping the test set completely unseen during training, potentially leading to overly optimistic performance metrics (data leakage). Instead, these techniques are applied *only to the training set* after the split.

3.6 Train-Test Split Strategy

The cleaned dataset was split into training and test sets using stratified sampling to preserve the original class distribution. An 80/20 split ratio was adopted:

Stratified splitting ensures that minority classes (e.g., Bot, Web Attack) are proportionally represented in both sets, which is essential for reliable evaluation of multi-class classifiers.

4 Methods

This section presents the machine learning pipeline from input to output, the algorithms employed, and the evaluation methodology. Figure 3 illustrates the overall process.

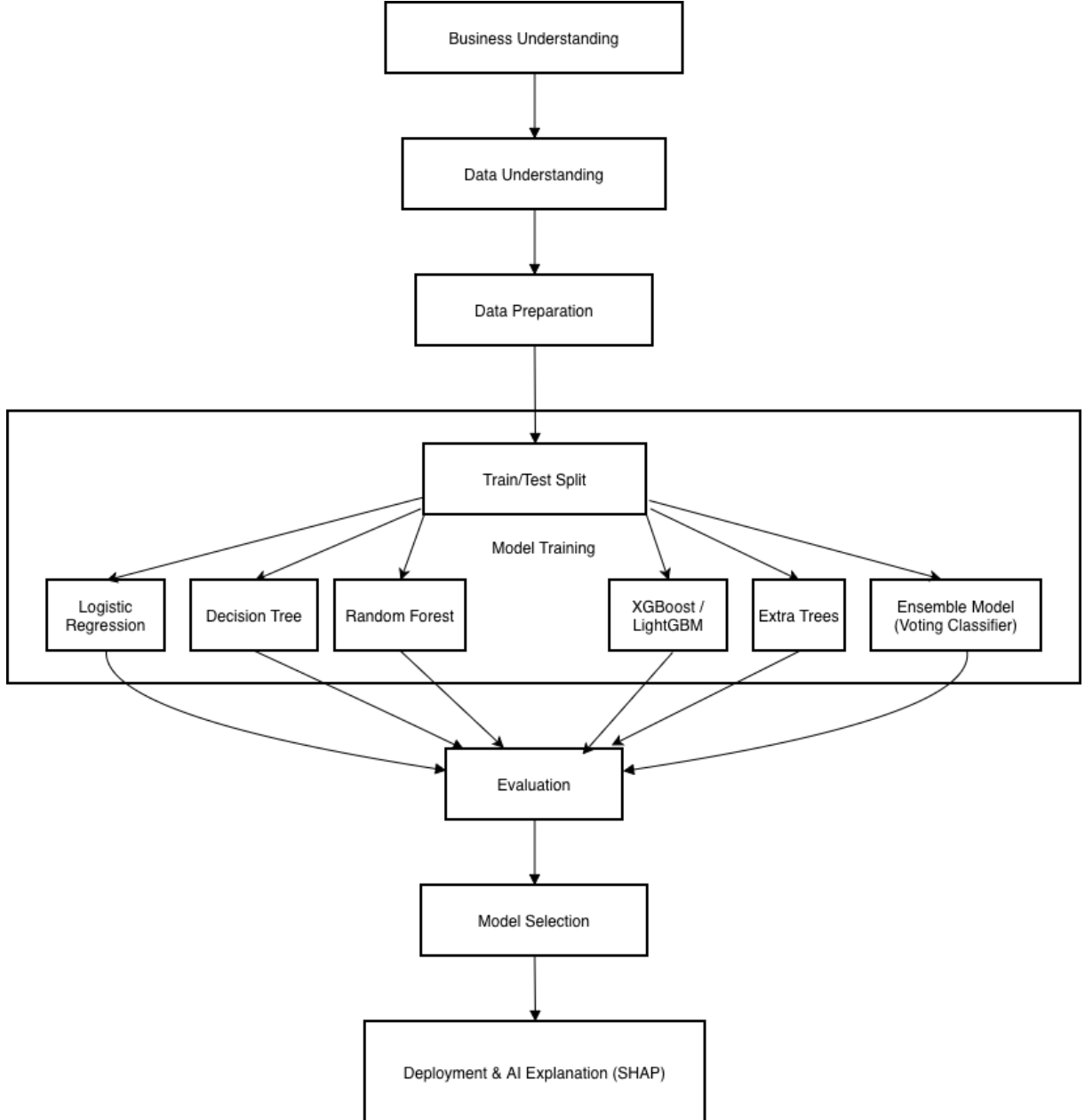


Figure 3: End-to-end machine learning pipeline for the NIDS project.

4.1 Experimental Setup

4.1.1 Data Splitting

The cleaned dataset (2,520,762 samples, 52 numerical features) was split into training (80%) and test (20%) sets using stratified sampling with a fixed random seed (42) for reproducibility.

Stratification ensures that minority classes (e.g., Heartbleed with 11 samples) are proportionally represented in both sets.

4.1.2 Evaluation Metrics

Given the severe class imbalance, relying solely on accuracy would be misleading—a model predicting only BENIGN would achieve 83% accuracy. The following metrics are employed:

- **Accuracy:** The fraction of correctly classified samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Precision (per class c):** The fraction of predicted positives that are truly positive:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \quad (2)$$

- **Recall (per class c):** The fraction of actual positives that are correctly identified:

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (3)$$

- **F1-score:** The harmonic mean of precision and recall:

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (4)$$

- **F1-macro:** The unweighted average of per-class F1 scores, treating all classes equally regardless of size:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c \quad (5)$$

This is the **primary metric** for model selection, as it is sensitive to minority class performance.

- **F1-weighted:** Weighted by class support, reflecting overall performance:

$$F1_{\text{weighted}} = \sum_{c=1}^C \frac{n_c}{N} \cdot F1_c \quad (6)$$

- **ROC-AUC (macro):** Area Under the Receiver Operating Characteristic Curve, averaged across classes using One-vs-Rest binarisation.
- **Matthews Correlation Coefficient (MCC):** A balanced measure that accounts for all four confusion matrix categories, ranging from -1 to $+1$:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (7)$$

4.2 Baseline Models

Six machine learning algorithms were trained as baseline models. Each is described below with its core mechanism.

4.2.1 Logistic Regression

Logistic Regression is a linear classifier that models the probability of class membership using the sigmoid function. For multi-class problems, the softmax (multinomial) extension is used:

$$P(y = c \mid \mathbf{x}) = \frac{e^{\mathbf{w}_c^T \mathbf{x} + b_c}}{\sum_{j=1}^C e^{\mathbf{w}_j^T \mathbf{x} + b_j}} \quad (8)$$

where $\mathbf{w}_c$ and b_c are the weight vector and bias for class c . The model is optimised using the L-BFGS solver with a maximum of 1,000 iterations. StandardScaler was applied to normalise features before training, as linear models are sensitive to feature scales.

4.2.2 Decision Tree

A Decision Tree recursively partitions the feature space by selecting the split that maximises *information gain*, measured using the Gini impurity:

$$G(t) = 1 - \sum_{c=1}^C p_c^2 \quad (9)$$

where p_c is the proportion of samples belonging to class c in node t . The tree was constrained with `max_depth=20` to prevent overfitting.

4.2.3 Random Forest

Random Forest is an ensemble of B decision trees trained on bootstrap samples (bagging), with random feature subsets at each split. The final prediction is determined by majority vote:

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_B(\mathbf{x})\} \quad (10)$$

where $h_b(\mathbf{x})$ is the prediction of the b -th tree. This procedure reduces variance compared to a single tree. The model was configured with $B = 300$ trees.

4.2.4 XGBoost (Extreme Gradient Boosting)

XGBoost builds an additive ensemble of trees sequentially, where each new tree fits the residual errors of the previous ensemble. The objective function at iteration t is:

$$\mathcal{L}^{(t)} = \sum_{i=1}^N l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (11)$$

where l is the loss function, f_t is the new tree, and $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2$ is the regularisation term (with T being the number of leaves and $\mathbf{w}$ the leaf weights). The loss is approximated using second-order Taylor expansion:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^N \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t) \quad (12)$$

where $g_i = \partial l / \partial \hat{y}_i^{(t-1)}$ and $h_i = \partial^2 l / \partial (\hat{y}_i^{(t-1)})^2$ are the first and second-order gradients. Class imbalance was addressed using computed sample weights based on inverse class frequency.

4.2.5 LightGBM

LightGBM is a gradient boosting framework that introduces two key optimisations:

- **Gradient-based One-Side Sampling (GOSS):** Retains all instances with large gradients and randomly samples from small-gradient instances, reducing computation while maintaining accuracy.
- **Exclusive Feature Bundling (EFB):** Bundles mutually exclusive features to reduce the effective number of features.

LightGBM uses *leaf-wise* tree growth (growing the leaf with the maximum loss reduction) rather than level-wise, producing deeper and more specialised trees. This makes it significantly faster than traditional gradient boosting, achieving a training time of 69.56s compared to XGBoost's 112.8s.

4.2.6 Extra Trees (Extremely Randomized Trees)

Extra Trees extends Random Forest by introducing additional randomness: instead of finding the optimal split threshold, it selects thresholds *randomly* for each candidate feature. This further reduces variance at the cost of slightly higher bias, and results in faster training (81.00s vs. Random Forest's 277.64s).

4.3 Hyperparameter Tuning

Hyperparameter tuning was performed on the top three baseline models (Random Forest, XGBoost, LightGBM) using **Randomised Search Cross-Validation** (RandomizedSearchCV) with 3-fold stratified CV. To reduce computational cost, a 20% stratified subsample of the training data was used for the search phase. Class imbalance was addressed through model-specific mechanisms: `class_weight='balanced'` for Random Forest and LightGBM, and inversely proportional sample weights for XGBoost.

Figure 4 details the hyperparameter search space explored for the Random Forest model.

Parameter	Values Explored
n_estimators	100, 200, 300, 500
max_depth	10, 20, 30, None
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 4
class_weight	balanced, balanced_subsample, None

Figure 4: Random Forest Search Space

Figure 5 presents the parameter grid used to tune the XGBoost classifier.

Parameter	Values Explored
n_estimators	100, 200, 300, 500
max_depth	3, 5, 7, 10
learning_rate	0.01, 0.05, 0.1, 0.2
subsample	0.6, 0.8, 1.0
colsample_bytree	0.6, 0.8, 1.0
min_child_weight	1, 3, 5

Figure 5: XGBoost Search Space

Figure 6 illustrates the specific hyperparameter variations tested for LightGBM.

Parameter	Values Explored
n_estimators	100, 200, 300, 500
num_leaves	15, 31, 63, 127
learning_rate	0.01, 0.05, 0.1, 0.2
min_child_samples	5, 10, 20, 50
subsample	0.6, 0.8, 1.0
colsample_bytree	0.6, 0.8, 1.0
class_weight	balanced, None

Figure 6: LightGBM Search Space

4.4 Ensemble Model

To explore whether combining multiple models could improve robustness, a **Soft Voting Classifier** was constructed using the three best baseline models: XGBoost, LightGBM, and Random Forest.

Soft Voting works by averaging the predicted probability distributions from each base model m :

$$\hat{P}(y = c \mid \mathbf{x}) = \frac{1}{M} \sum_{m=1}^M P_m(y = c \mid \mathbf{x}) \quad (13)$$

The final prediction is the class with the highest average probability:

$$\hat{y} = \arg \max_c \hat{P}(y = c \mid \mathbf{x}) \quad (14)$$

This approach is more nuanced than hard voting (majority rule) because it leverages the confidence levels of each model.

Figure 7 depicts the configuration of the Soft Voting Classifier combining the top three base learners.

Component	Configuration
Base model 1	XGBoost (n_estimators=300, max_depth=7)
Base model 2	LightGBM (n_estimators=300, num_leaves=31)
Base model 3	Random Forest (n_estimators=300)
Voting method	Soft (probability averaging)

Figure 7: Ensemble configuration for each models.

4.5 Deployment

The final selected model was serialised using `joblib` and deployed as an interactive web dashboard built with **Streamlit** (app/app.py). The deployment artefacts include:

- `models/xgboost_best_model.joblib` — the trained XGBoost classifier.
- `models/label_encoder.joblib` — the label encoder used to map integer predictions back to attack-type names.
- `scripts/generate_samples.py` — a utility script that extracts representative sample network flows from the test set for dashboard demonstration.

The dashboard allows users to upload a CSV file containing network flow features (or use pre-loaded sample data) and receive real-time classification results with prediction probabilities. SHAP (SHapley Additive exPlanations) values are computed to provide feature-level explanations for each prediction, enhancing the interpretability of the model's decisions.

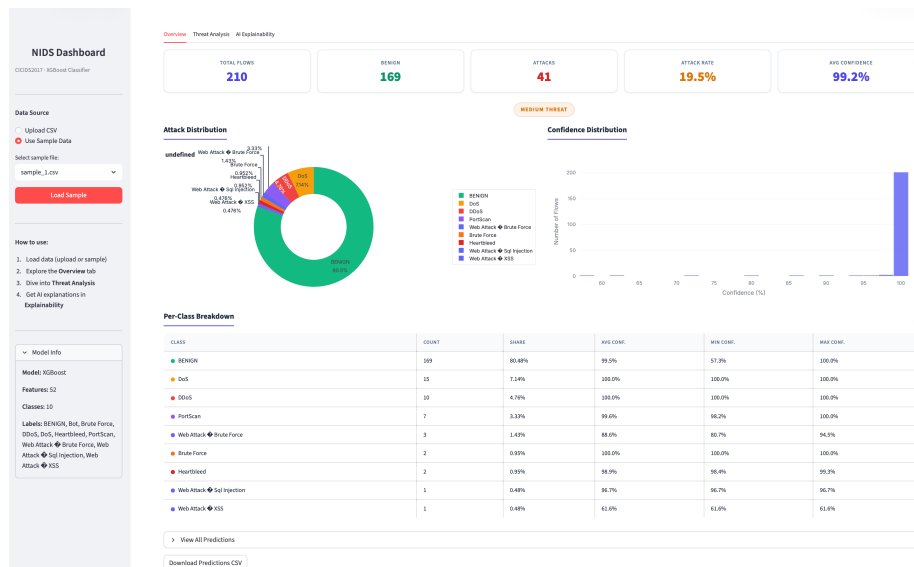


Figure 8: Streamlit Dashboard: Overview

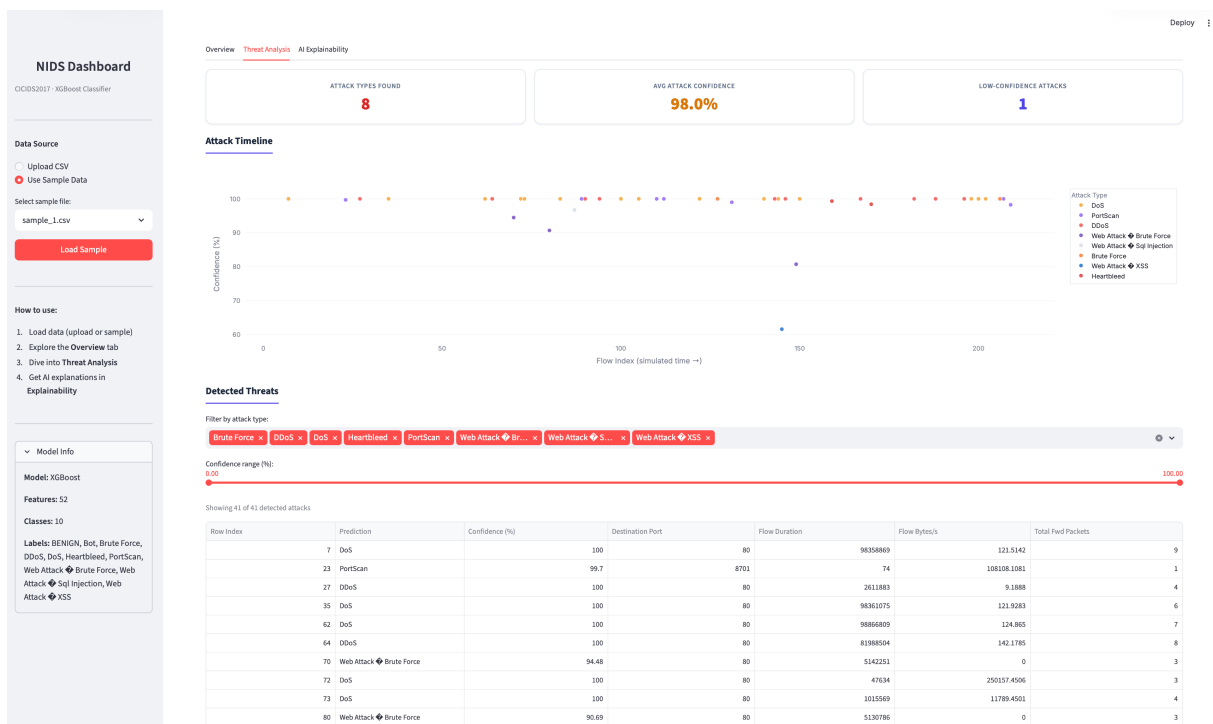


Figure 9: Streamlit Dashboard: Threat Analysis

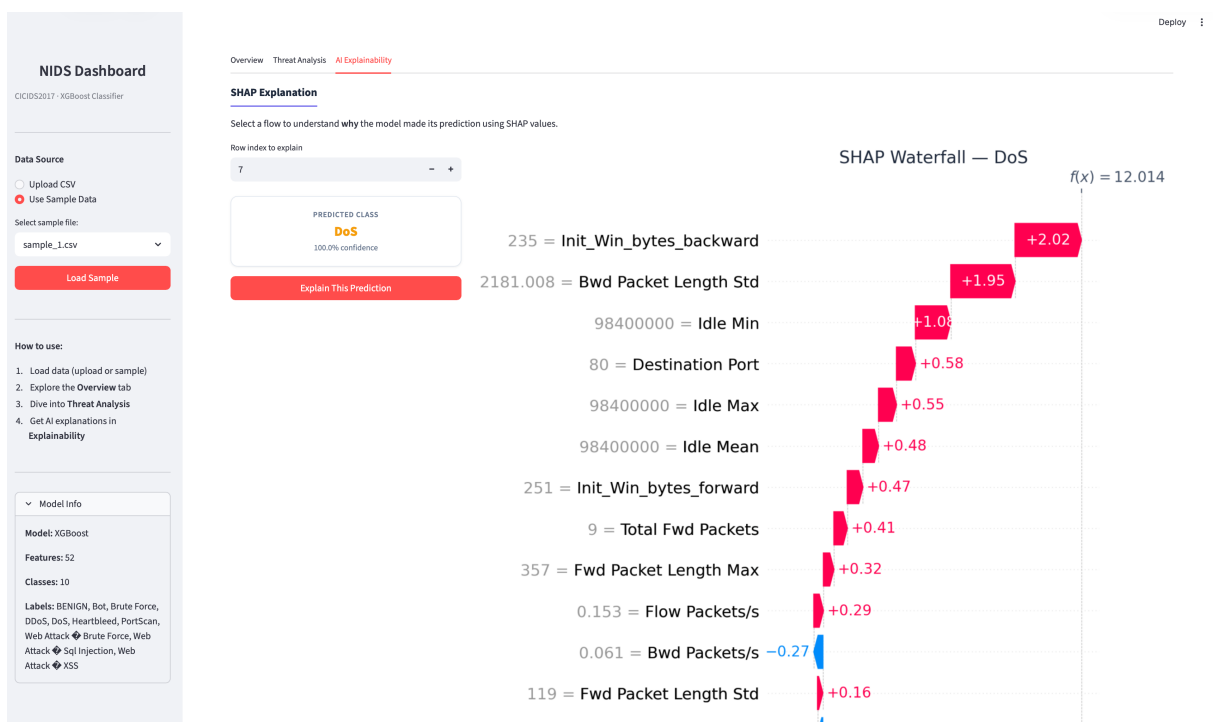


Figure 10: Streamlit Dashboard: Explainable AI (SHAP)

5 Results and Discussion

5.1 Baseline Model Comparison

All six baseline models were trained on the full training set and evaluated on the held-out test set. Table 3 summarizes the key performance metrics.

Table 3: Baseline model performance comparison on the CICIDS2017 test set.

Model	Accuracy	F1-macro	F1-weighted	MCC	Train Time
Logistic Regression	97.62%	0.567	0.976	0.921	113.92s
Decision Tree	99.83%	0.784	0.998	0.994	72.65s
Random Forest	99.82%	0.827	0.998	0.994	277.64s
XGBoost	99.87%	0.859	0.999	0.996	112.80s
LightGBM	99.89%	0.830	0.999	0.996	69.56s
Extra Trees	99.78%	0.813	0.998	0.992	81.00s

Figure 11 provides a visual comparison of the F1-macro scores across all six baseline models.

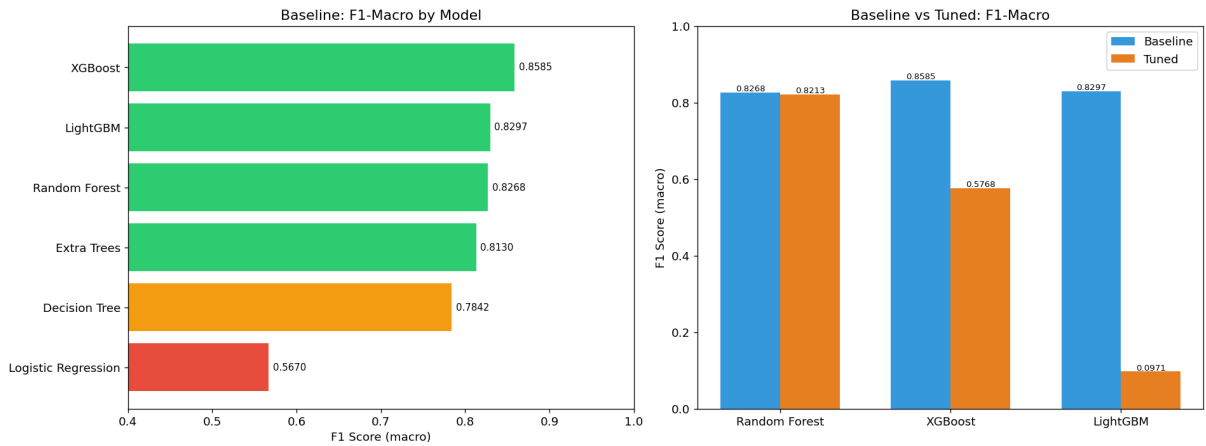


Figure 11: F1-Score by model results bar chart.

Key observations:

1. All tree-based models achieve exceptionally high weighted metrics ($>99.7\%$), reflecting strong overall classification. However, macro metrics reveal significant differences in minority class handling.
2. XGBoost leads with F1-macro of 0.859, followed by LightGBM (0.830) and Random Forest (0.827).
3. Logistic Regression lags significantly (F1-macro: 0.567), confirming that this multi-class IDS problem requires non-linear decision boundaries.

- XGBoost achieves a perfect ROC-AUC (macro) of 1.0, indicating excellent class separation in probability space.

The detailed classification reports and confusion matrices for each individual model are shown below.

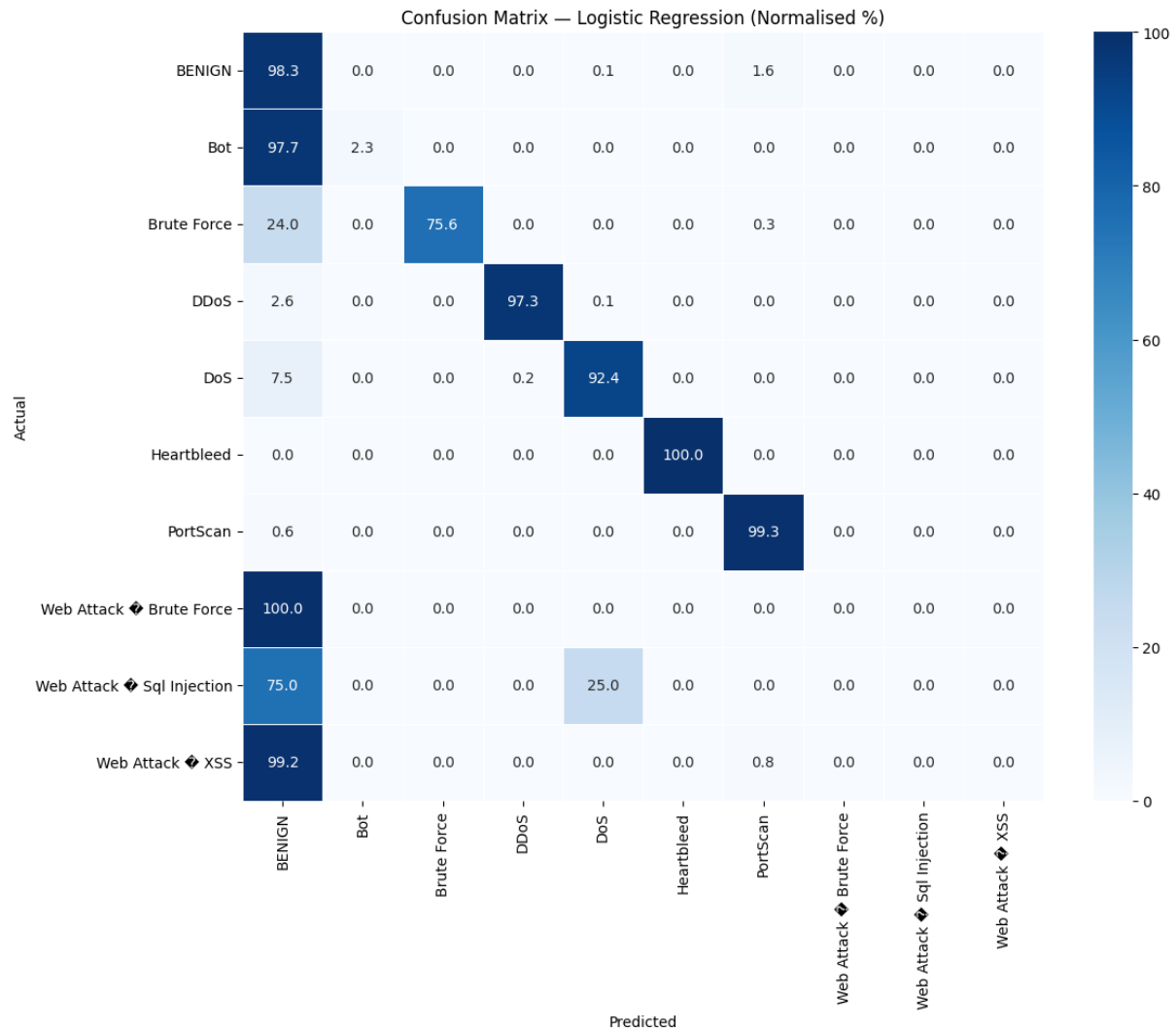


Figure 12: Confusion Matrix for Logistic Regression model.

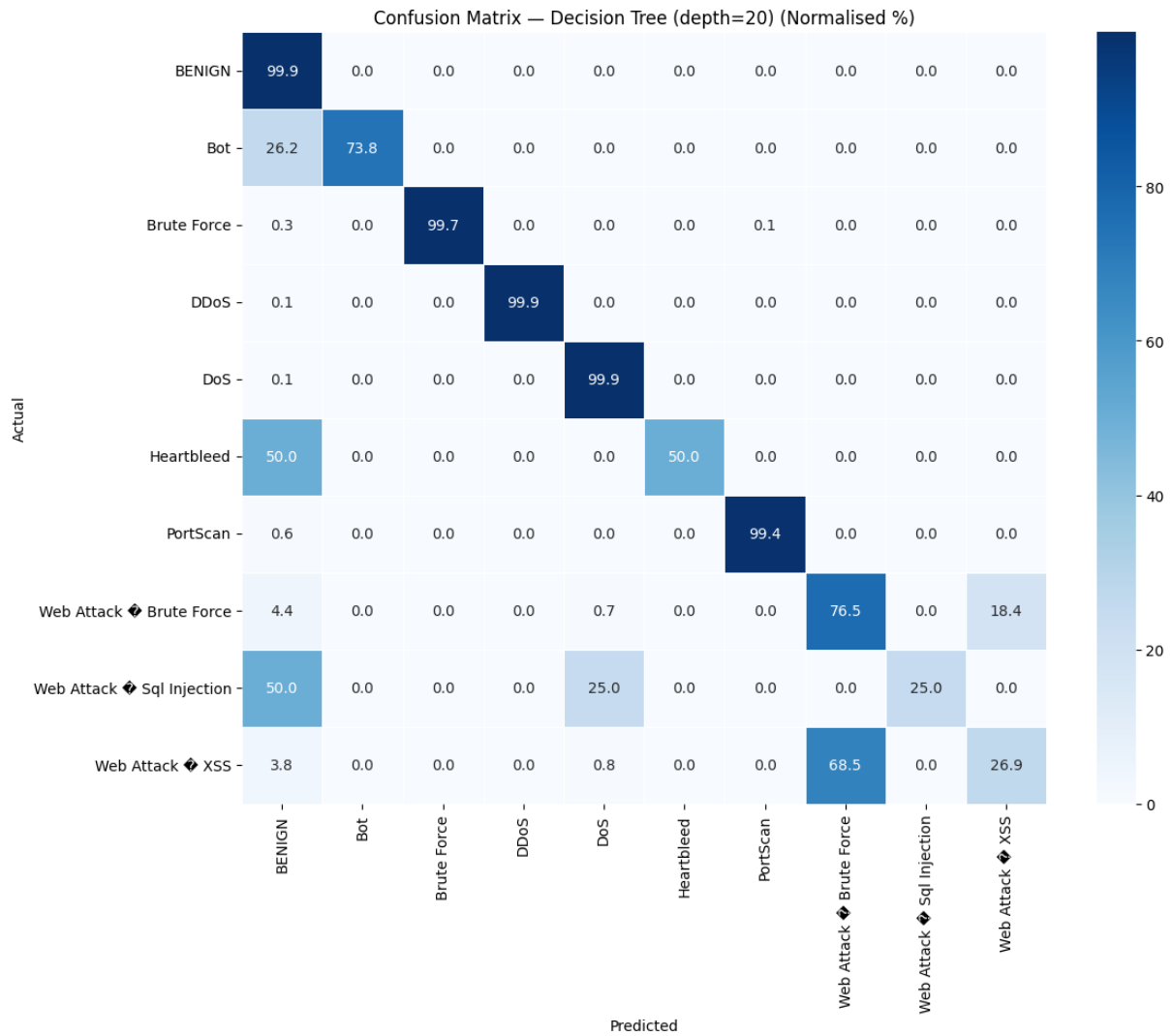


Figure 13: Confusion Matrix for Decision Tree model.

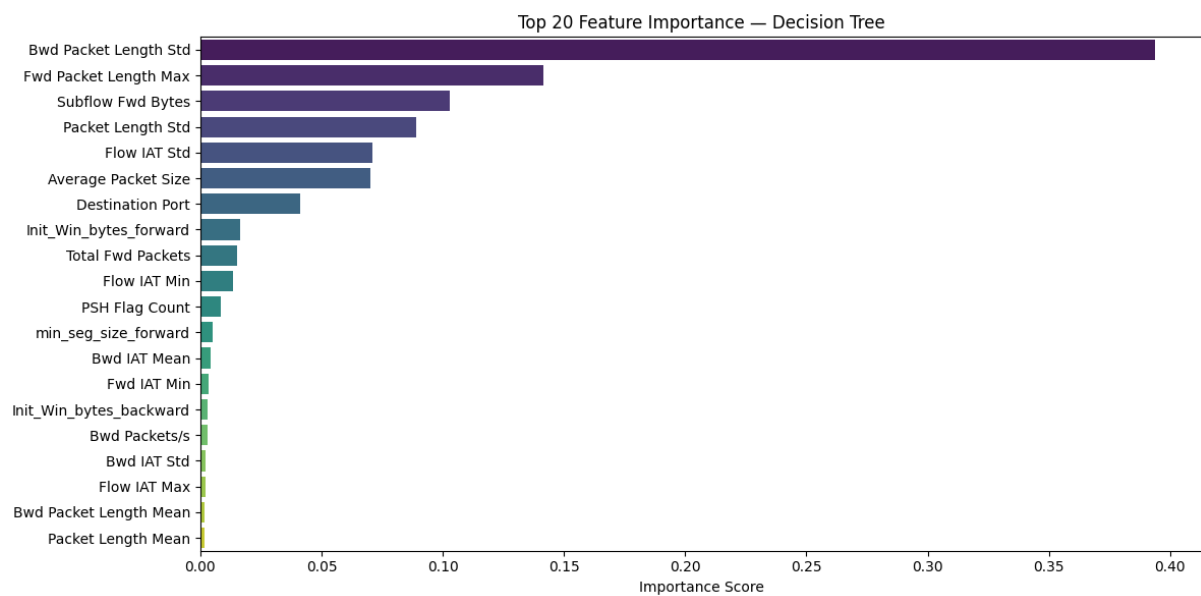


Figure 14: Feature Important chart for Decision Tree model.

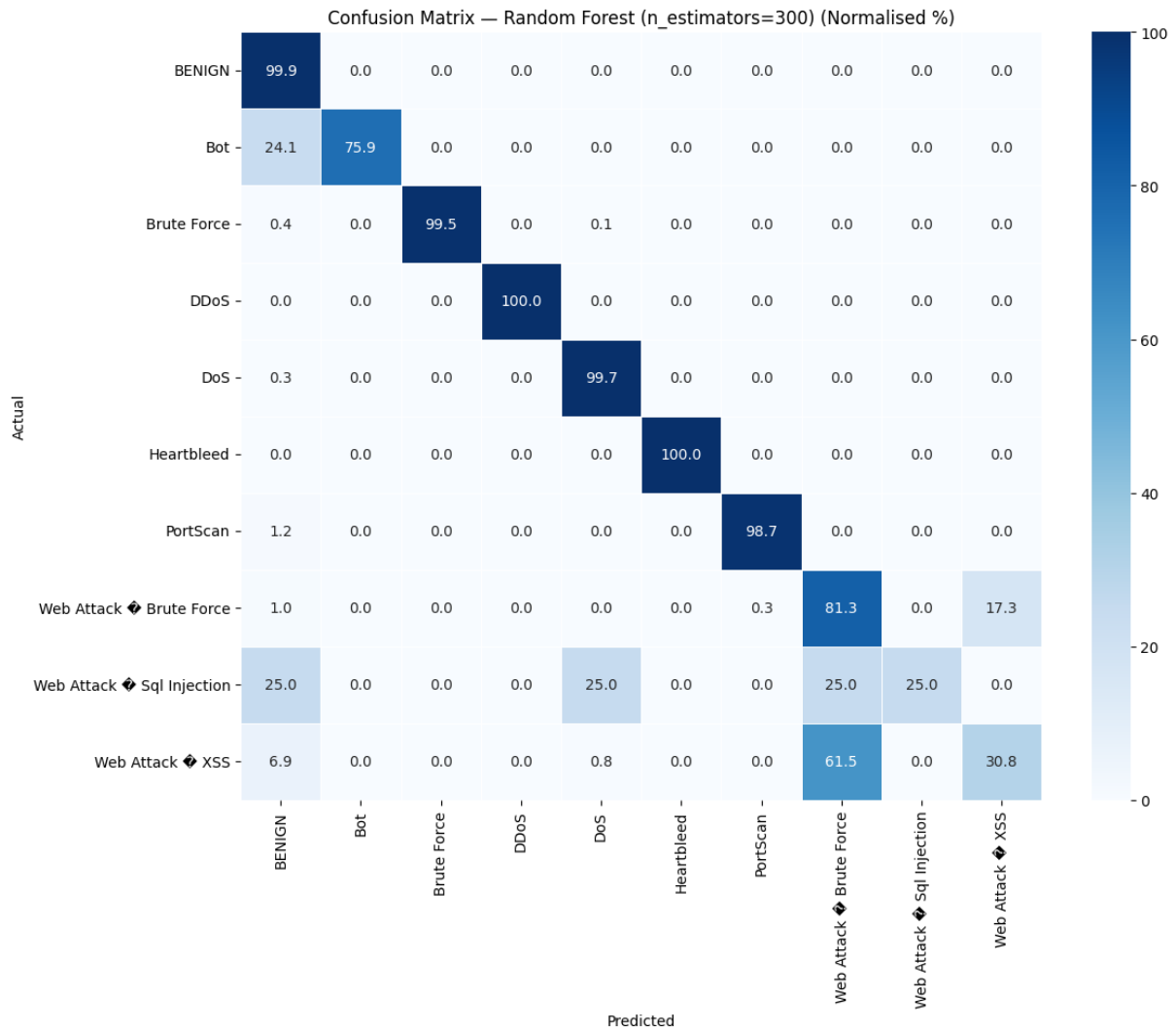


Figure 15: Confusion Matrix for Random Forest model.

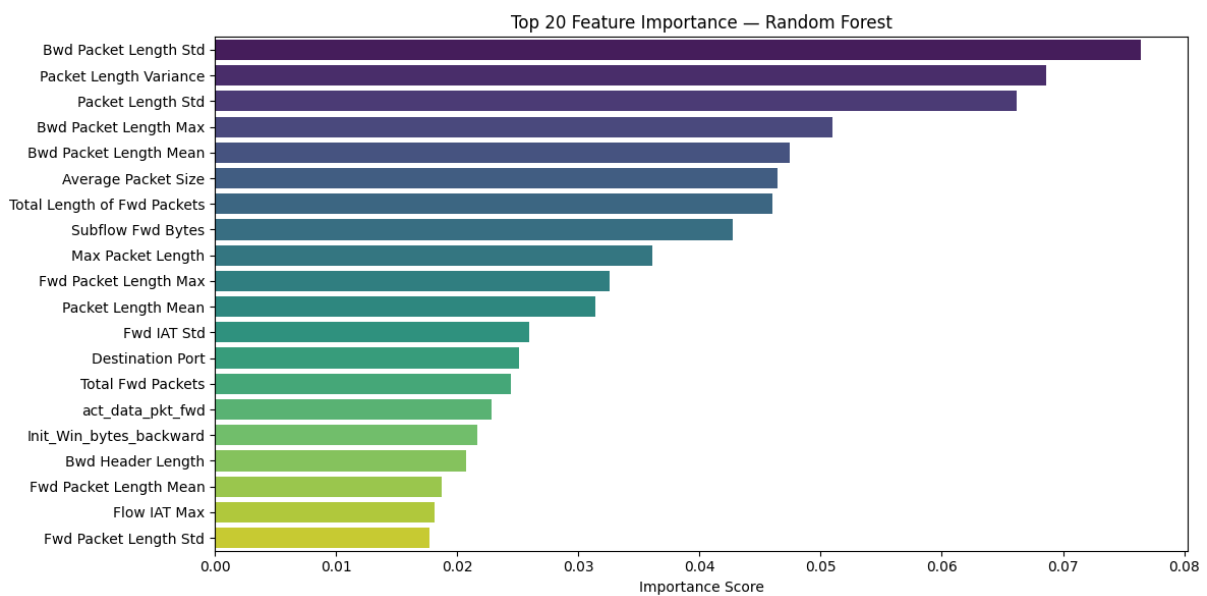


Figure 16: Feature Important chart for Random Forest model.

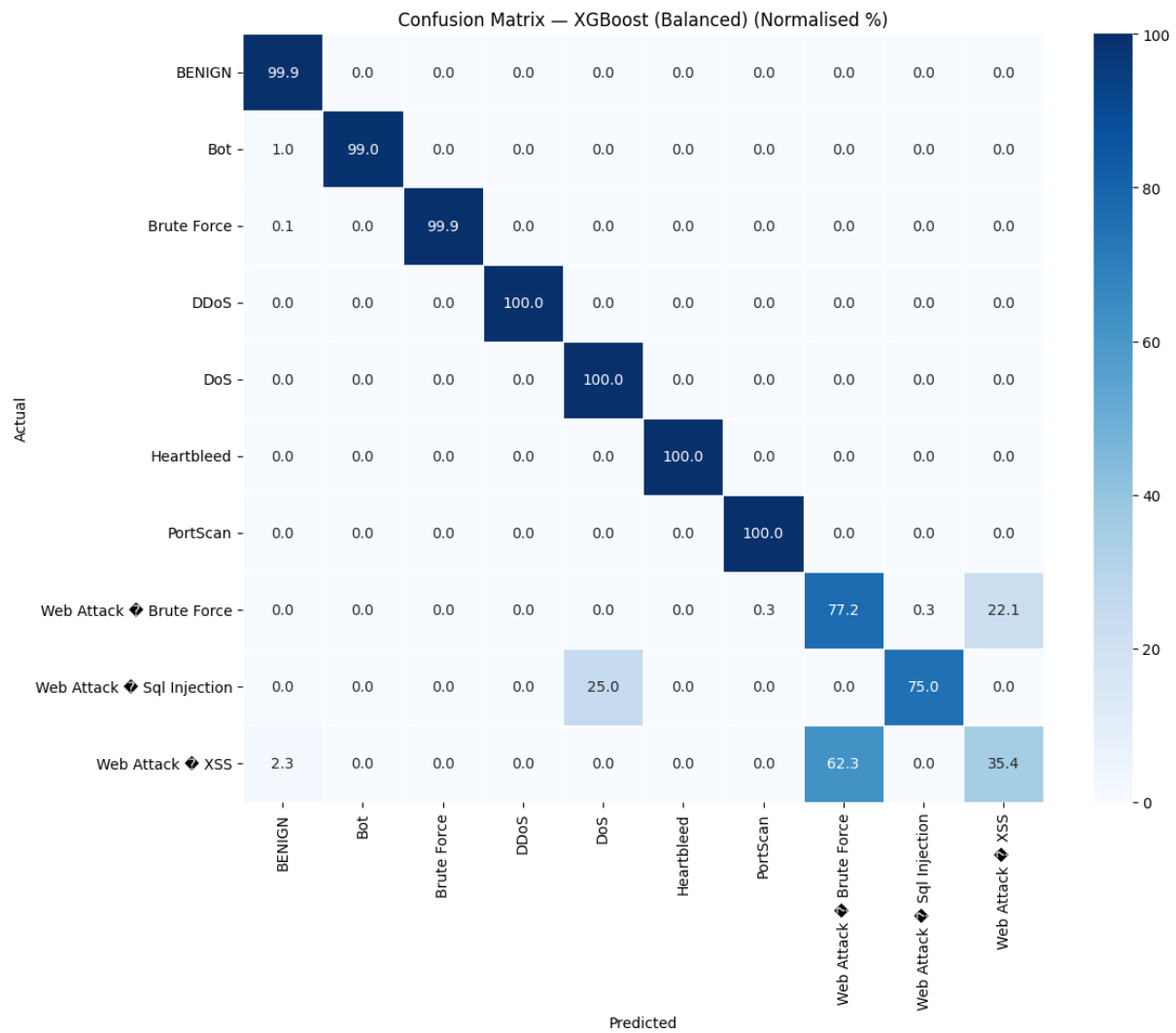


Figure 17: Confusion Matrix for XGBoost model.

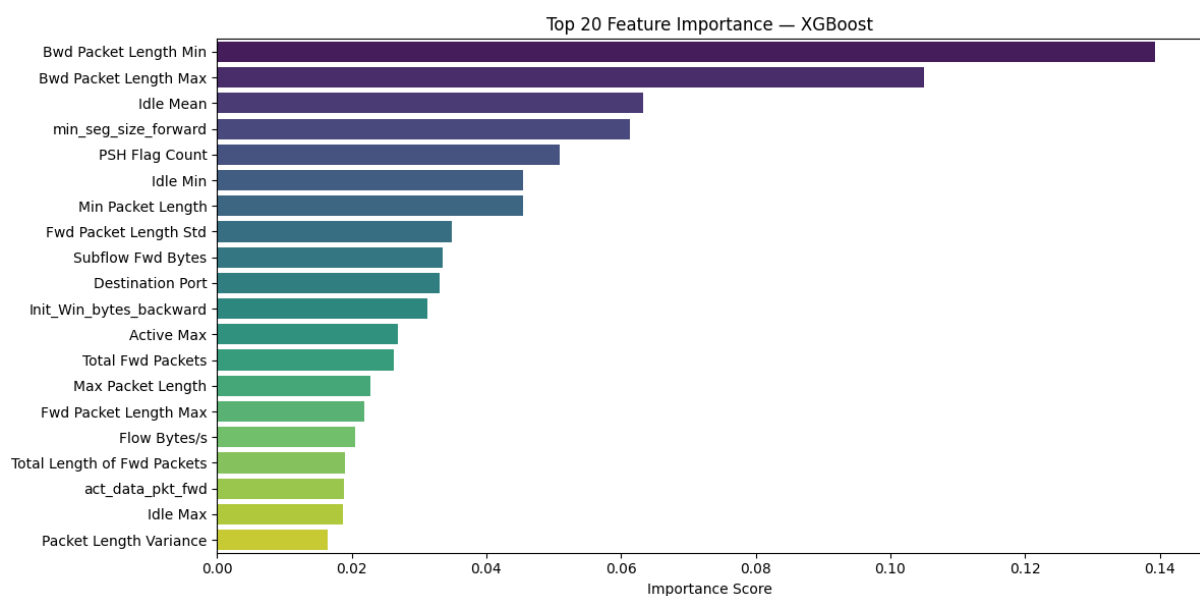


Figure 18: Feature Important chart for XGBoost model.

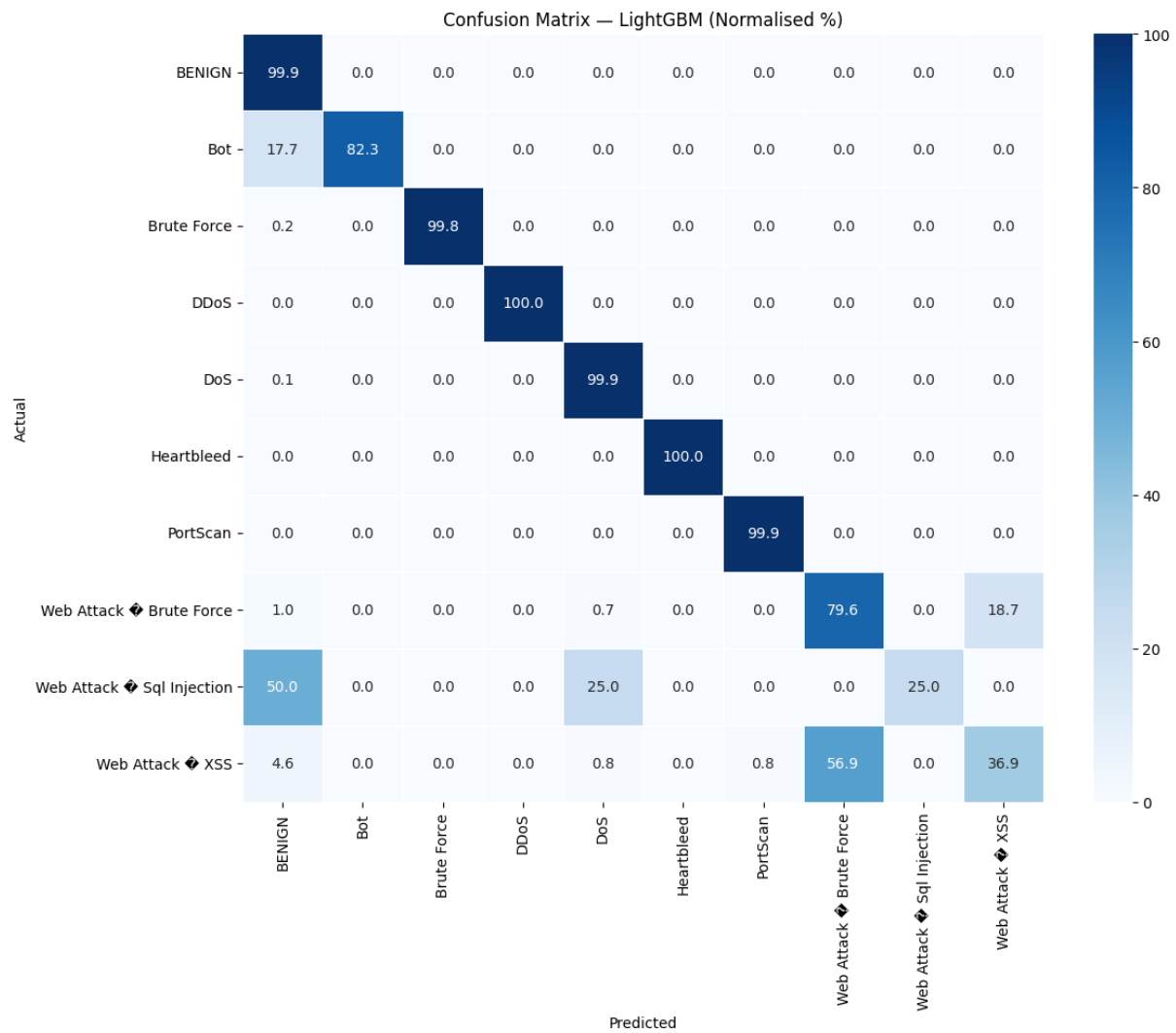


Figure 19: Confusion Matrix for LightGBM model.

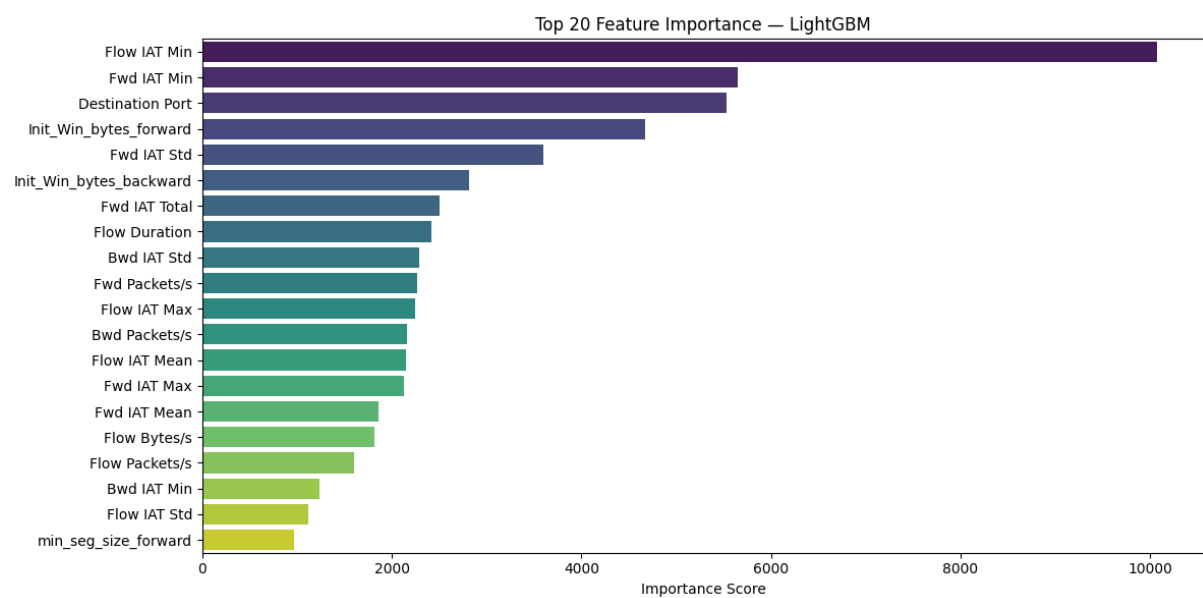


Figure 20: Feature Important chart for LightGBM model.



Figure 21: Confusion Matrix for Extra Trees model.

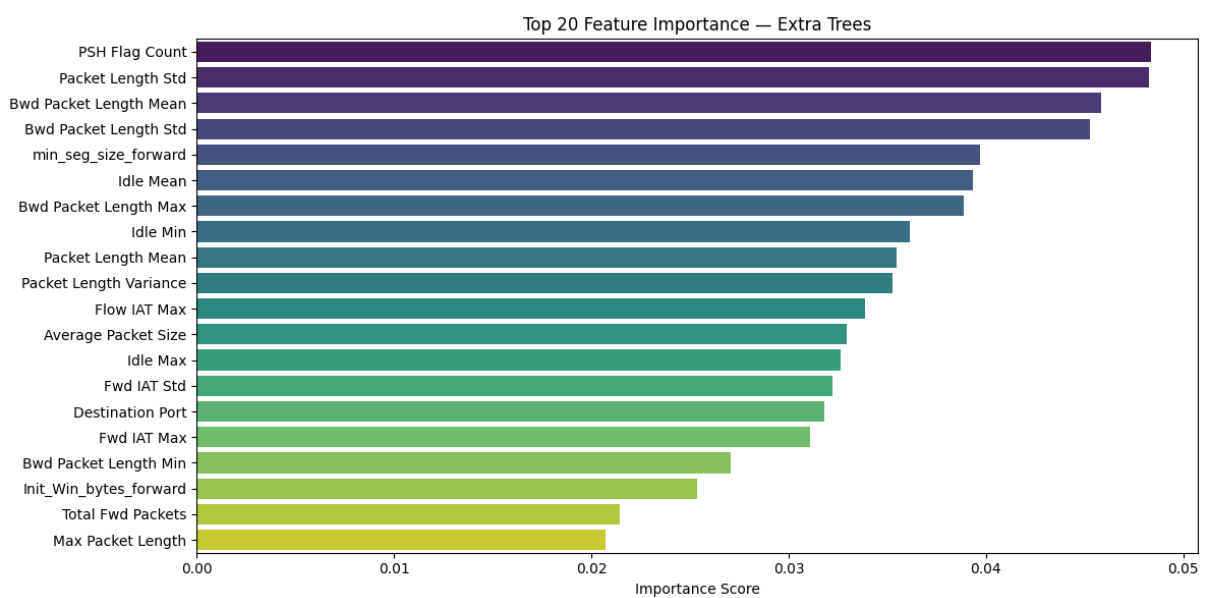


Figure 22: Feature Important chart for Extra Trees model.

5.2 Hyperparameter Tuning Results

Unexpectedly, hyperparameter tuning *degraded* performance for XGBoost and LightGBM, while Random Forest remained approximately unchanged. This counter-intuitive result is attributed to three factors:

1. **Extreme minority class noise in cross-validation:** The stratified subsample contained only 2 Heartbleed and 4 SQL Injection samples. With 3-fold CV, individual folds could contain 0 or 1 samples from these classes, causing the F1-macro optimisation target to become highly unstable.
2. **Over-aggressive class rebalancing:** When `class_weight='balanced'` is applied to extremely rare classes (e.g., Heartbleed with 2 samples in the subsample), the model assigns disproportionately large weights to these samples. LightGBM's tuned version collapsed to F1-macro of 0.097, with accuracy dropping to 68% due to massive over-prediction of minority classes.
3. **Suboptimal parameter combinations:** XGBoost's best-found parameters included `learning_rate=0.01` with only `n_estimators=100`, which is an underfitting configuration requiring $\geq 1,000$ estimators to converge.

Key Takeaway: When dealing with extreme class imbalance (classes with < 20 samples), aggressive rebalancing techniques can do more harm than good. The baseline models, which used moderate or no rebalancing, achieved better performance.

Figure 23 visually contrasts the performance of the models before and after hyperparameter tuning.

Model	F1-macro (Baseline)	F1-macro (Tuned)	Δ	Verdict
Random Forest	0.827	0.821	-0.006	Comparable
XGBoost	0.859	0.577	-0.282	Degraded
LightGBM	0.830	0.097	-0.733	Collapsed

Figure 23: Tuned vs Baseline Performance.

5.3 Ensemble Model Results

The Soft Voting Classifier (XGBoost + LightGBM + Random Forest) was trained on the full training set and evaluated on the test set. The ensemble achieved marginally higher metrics compared to XGBoost alone, validating that combining diverse models can reduce individual

model weaknesses. However, the improvement is small in magnitude.

Figure 24 shows the resulting confusion matrix for the Soft Voting ensemble model.

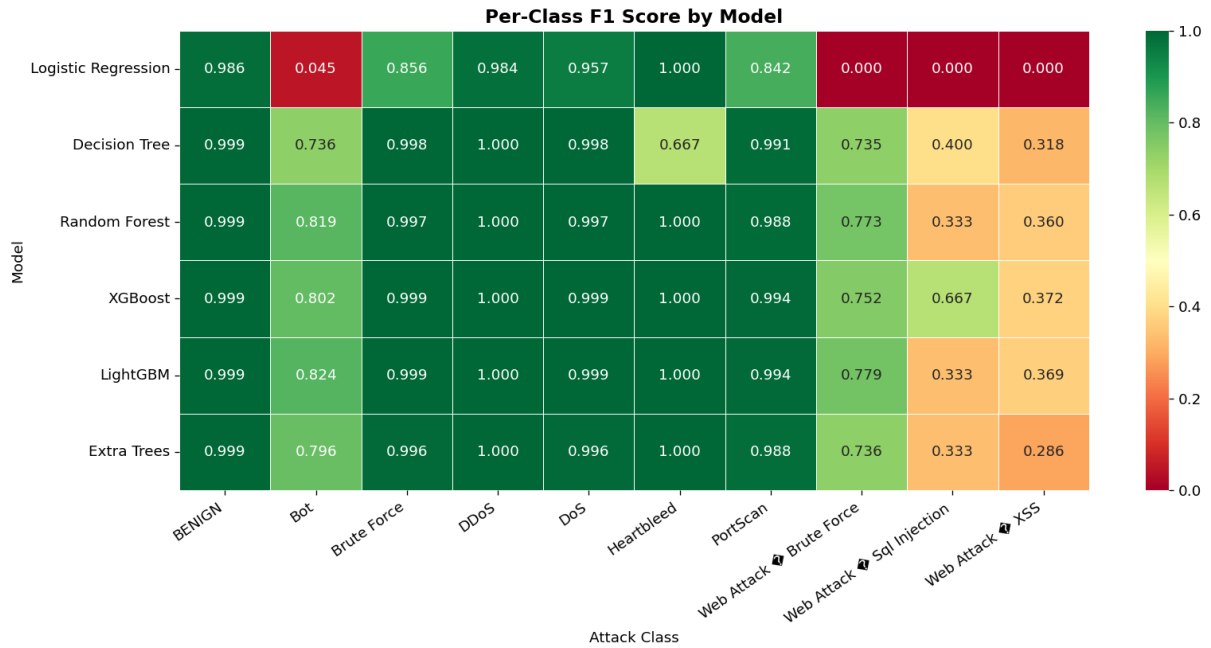


Figure 24: Confusion Matrix for Ensemble model.

5.4 Per-Class Performance Analysis

To understand how each model handles individual attack types, per-class F1 scores were computed across all six baseline models.

Figure ?? breaks down the F1-scores per class across the evaluated models.



Key observations:

1. **High-volume classes** (BENIGN, DDoS, DoS, Brute Force): All tree-based models achieve near-perfect F1 scores (>0.99). These classes have sufficient training data for effective learning.
2. **Medium-volume classes** (PortScan, Bot): Performance ranges from 0.74 to 0.82 F1. Bot detection is particularly challenging due to its low sample count (1,948 total) and traffic patterns that can mimic benign behaviour.
3. **Low-volume classes** (Web Attack sub-types): These are the most challenging. Web Attack – XSS ($F1 \approx 0.3$) and Web Attack – SQL Injection ($F1 \approx 0.3\text{--}0.7$) suffer from extremely limited training data. The high variability in SQL Injection F1 (0.000 to 0.667) reflects the statistical instability of evaluating models on just 4 test samples.
4. **Heartbleed**: Despite having only 11 total samples (2 test), most models achieve $F1 = 1.0$. This suggests Heartbleed traffic has a very distinctive network pattern, but the result should be interpreted with extreme caution given the tiny sample size.

5.5 Error Analysis

Detailed error analysis was performed on the best-performing model (XGBoost baseline) to identify systematic misclassification patterns.

Figure 25 highlights the most common misclassification pairs made by the selected XGBoost model.

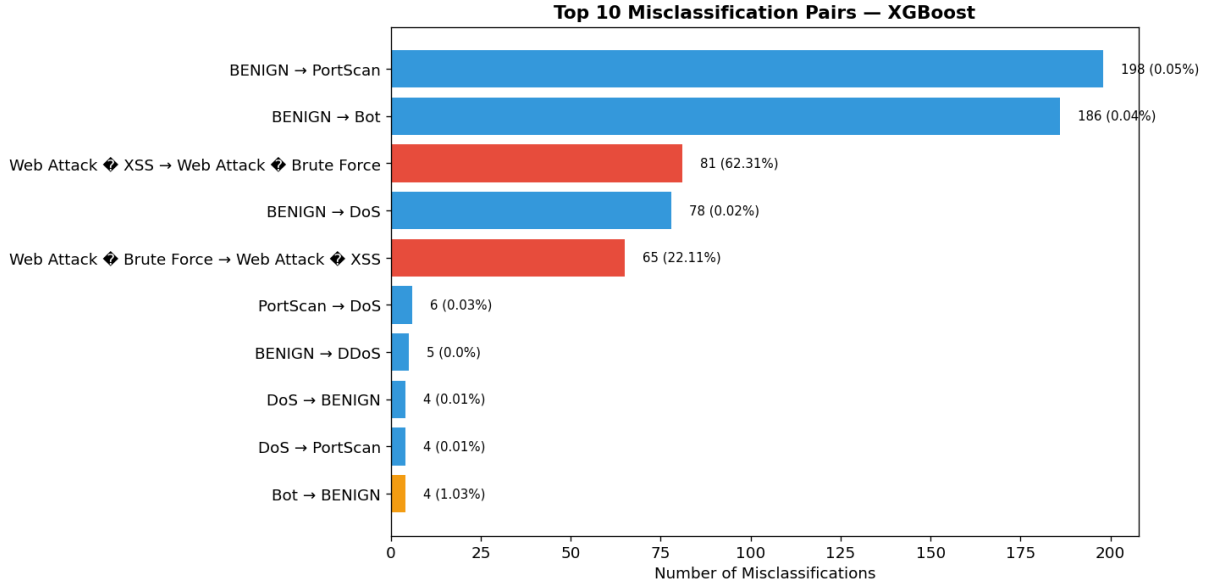


Figure 25: Top Misclassification Pairs bar chart.

1. **Web Attack cross-confusion:** The most striking pattern is the mutual confusion between Web Attack – XSS and Web Attack – Brute Force. 62.3% of XSS attacks are misclassified as Brute Force, and 22.1% of Brute Force are misclassified as XSS. This occurs because both attack types target web applications and share similar network-level features (HTTP traffic patterns, request rates, packet sizes). Distinguishing between them would likely require application-layer features (e.g., payload content) that are not present in the CICIDS2017 feature set.
2. **BENIGN false positives:** A small number of benign samples are misclassified as PortScan (198), Bot (186), and DoS (78). In absolute terms these numbers appear significant, but relative to the 419,012 benign test samples, they represent a false positive rate of only 0.11% — approximately 1 false alert per 900 benign connections, which is generally acceptable in production IDS deployments.
3. **Low total errors:** For XGBoost, the total number of misclassifications across all classes is approximately 700 out of 504,153 test samples, yielding an overall error rate of 0.13%. The errors are concentrated in the extremely rare attack types where training data is insufficient.

5.6 Model Selection

5.6.1 Selection Criteria

The final model was selected based on a multi-criteria ranking across four key metrics: F1-macro, F1-weighted, ROC-AUC (macro), and MCC. Each model was ranked 1 (best) to 6 (worst) for each metric, and the average rank determined the final ordering.

XGBoost and LightGBM were tied in average rank (1.50). **XGBoost was selected** as the final deployment model for the following reasons:

1. **Higher F1-macro (0.859 vs. 0.830):** XGBoost is better at detecting minority attack classes, which is the primary objective of an intrusion detection system. Missing an attack (false negative) is more dangerous than a false alarm (false positive).
2. **Perfect ROC-AUC (1.0):** XGBoost demonstrates perfect class separation in probability space, meaning that with appropriate threshold tuning, detection rates can be further optimised for specific deployment scenarios.
3. **Reasonable training time (112.8s):** While not the fastest model, XGBoost's training time is practical for periodic retraining in production environments.

5.6.2 Why Not the Ensemble Model?

Although the Voting Ensemble achieved marginally higher metrics than XGBoost alone, it was not selected for deployment due to practical considerations:

- **Model complexity:** The ensemble contains three separate models, resulting in a significantly larger model file and higher memory requirements.
- **Inference latency:** In network intrusion detection, real-time prediction speed is critical. The ensemble requires running all three models and averaging their outputs, tripling inference time.
- **Marginal improvement:** The performance gain does not justify the increased complexity and operational cost. In production IDS deployments, the trade-off favours the simpler, faster single model.

The trained XGBoost model was serialised and saved at `models/xgboost_best_model.joblib` for deployment.

6 Conclusion and Perspectives

This project comprehensively addressed the multi-class network intrusion detection problem on the CICIDS2017 dataset through a rigorous end-to-end machine learning pipeline. The key findings are:

1. **Non-linear models are essential:** Logistic Regression’s significantly lower performance (F1-macro: 0.567) compared to tree-based models (>0.78) confirms that network intrusion patterns require non-linear decision boundaries that cannot be captured by linear classifiers.
2. **Gradient boosting outperforms bagging:** XGBoost (0.859) and LightGBM (0.830) outperform Random Forest (0.827) and Extra Trees (0.813) in F1-macro, demonstrating that sequential error correction is more effective than parallel averaging for this imbalanced dataset.
3. **Aggressive class rebalancing can backfire:** Hyperparameter tuning with `class_weight='balanced'` degraded all three tuned models, with LightGBM collapsing to F1-macro 0.097. Extreme minority classes (Heartbleed: 9 training samples, SQL Injection: 17 training samples) make balanced weighting unstable.
4. **Data limitation is the primary bottleneck:** The remaining performance gap (particularly on Web Attack classes, $F1 \approx 0.3$) cannot be solved by model selection or tuning alone. Improving detection of these rare attacks would require either more labelled samples or additional engineered features capturing application-layer behaviour.
5. **Selected model:** XGBoost (baseline) achieves the best balance of minority class detection (F1-macro: 0.859), overall accuracy (99.87%), and practical deployability.

Limitations of this work include: (i) the CICIDS2017 dataset, while comprehensive, was generated in a controlled lab environment and may not fully represent real-world network diversity; (ii) the feature set is limited to network-flow-level statistics and does not include payload or application-layer information; (iii) extremely rare attack classes (Heartbleed, SQL Injection) have insufficient samples for reliable evaluation.

Future perspectives: If given more time and computational resources, several directions could be explored:

- Augmenting the dataset with additional IDS benchmarks (e.g., UNSW-NB15, NSL-KDD) to improve minority class representation.
- Implementing online/incremental learning to adapt to evolving attack patterns in real-time.
- Enhancing the SHAP-based explainability module to provide actionable security insights for network administrators.

References

- [1] Steve Morgan, “Cybercrime To Cost The World 10.5 Trillion Annually By 2025,” 2025. [Online]. Available: <https://cybersecurityventures.com/hackerpocalypse-cybercrime-report-2016/>
- [2] Yang Guo, “A review of Machine Learning-based zero-day attack detection: Challenges and future directions,” 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0140366422004248>
- [3] A. H. Ali et al., “Unveiling machine learning strategies and considerations in intrusion detection systems: a comprehensive survey,” 2024. [Online]. Available: <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2024.1387354/full>
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” 2018. [Online]. Available: <https://www.scitepress.org/papers/2018/66398/66398.pdf>
- [5] Majed S. Alsayfi, “Interpretable Machine Learning Framework for Anomaly Detection in Intrusion Detection Systems using XAI Techniques,” 2025. [Online]. Available: <https://ijamjournal.org/ijam/publication/index.php/ijam/article/view/667>
- [6] Oyelakin A.M., Akanbi M.B., Ogundele T.S., Akanni A.O., Gbolagade M.D., and Rilwan M.D., “A Learning Approach for The Identification of Network Intrusions Based on Ensemble XGBoost,” 2023. [Online]. Available: <https://jurnal.yoctobrain.org/index.php/ijodas/article/view/88/167>
- [7] Vincent Zibi Mohale and Ibidun Christiana Obagbuwa, “Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability,” 2025. [Online]. Available: <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1520741/full>
- [8] Yashovardhan Sharma, Simon Birnbach, and Ivan Martinovic, “RADAR: A TTP-based Extensible, Explainable, and Effective System for Network Traffic Analysis and Malware Detection,” 2023. [Online]. Available: <https://arxiv.org/pdf/2212.03793>

A Project Plan Management

Table 4: Project timeline and task allocation.

Task Name	Priority	Owner	Start Date	End Date	Status	Issues
Problem Setup	High	All	10/01/2026	12/01/2026	Done	None
Literature review	Medium	Do Anh Thu	10/01/2026	18/01/2026	Done	None
EDA	High	Le Trung Hieu	19/01/2026	24/01/2026	Done	None
Data Preprocessing	High	Vu Ngoc Hai Dang	25/01/2026	31/01/2026	Done	None
Feature engineering	High	Nguyen Hoang An	01/02/2026	05/02/2026	Done	None
Data splitting	High	Le Trung Kien	06/02/2026	07/02/2026	Done	None
Baseline Model: Logistic Regression	High	Do Anh Thu	08/02/2026	12/02/2026	Done	None
Baseline Model: Decision Tree	High	Do Anh Thu	08/02/2026	12/02/2026	Done	None
Tree-Based Model: Random Forest	High	Le Trung Hieu	13/02/2026	20/02/2026	Done	None
Tree-Based Model: Extra Trees	High	Nguyen Hoang An	13/02/2026	20/02/2026	Done	None
Tree-Based Model: XGBoost	High	Vu Ngoc Hai Dang	13/02/2026	20/02/2026	Done	None
Tree-Based Model: LightGBM	High	Le Trung Kien	13/02/2026	20/02/2026	Done	None
Hyperparameter Tuning	Medium	Nguyen Hoang An	21/02/2026	26/02/2026	Done	None
Imbalance Handling	Medium	Nguyen Hoang An	27/02/2026	01/03/2026	Done	None
Model Evaluation	High	Le Trung Kien	02/03/2026	05/03/2026	Done	None
Model Selection	High	All	06/03/2026	07/03/2026	Done	None
Error Analysis	Medium	Le Trung Hieu	08/03/2026	10/03/2026	Done	None
Deployment	Medium	Vu Ngoc Hai Dang	11/03/2026	14/03/2026	Done	None
XAI Integration	Medium	Nguyen Hoang An	11/03/2026	15/03/2026	Done	None
Reporting & Presentation	High	All	16/03/2026	18/03/2026	Done	None

B Source Code & Data

The complete source code and dataset for this project are available at:

- **GitHub Repository:** <https://github.com/annghoangg/network-intrusion-detection-system>
- **Dataset:** The CICIDS2017 dataset is publicly available on Kaggle: <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset/>
- **Project Pipeline shortened:** https://drive.google.com/file/d/183XjTwRZr-KiwPTc3AxyIf0m8zcL0ovo/view?usp=share_link
- **Project Pipeline full:** <https://drive.google.com/file/d/1miDZyRl9wCciHPR90IJYU0-PoqMrEoai/view?usp=sharing>
- **Trained Model:** The final XGBoost model is stored at `models/xgboost_best_model.joblib` in the repository.